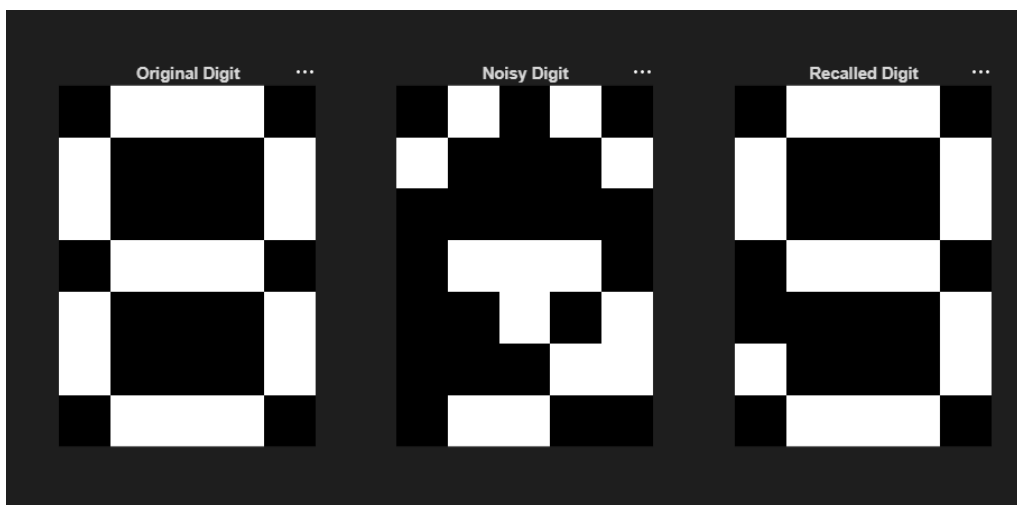


PatternStorageof10DigitsusingaDiscreteHopfieldNetwork.m × +		
C:\Users\Afroz\Desktop\NN\EXP10\Code\PatternStorageof10DigitsusingaDiscreteHopfieldNetwork.m		
1	clc;	50
2	clear;	51
3	close all;	52
4		53
5	%% Pattern Storage of 10 Digits using Discrete Hopfield Network	54
6	% 7 x 5 patterns for digits 0 to 9	55
7		56
8	%% Define digit patterns	57
9		58
10	D{1} = [	59
11	0 1 1 1 0;	60
12	1 0 0 0 1;	61
13	1 0 0 1 1;	62
14	1 0 1 0 1;	63
15	1 1 0 0 1;	64
16	1 0 0 0 1;	65
17	0 1 1 1 0	66
18	];	67
19		68
20	D{2} = [	69
21	0 0 1 0 0;	70
22	0 1 1 0 0;	71
23	0 0 1 0 0;	72
24	0 0 1 0 0;	73
25	0 0 1 0 0;	74
26	0 0 1 0 0;	75
27	0 1 1 1 0	76
28	];	77
29		78
30	D{3} = [	79
31	0 1 1 1 0;	80
32	1 0 0 0 1;	81
33	0 0 0 0 1;	82
34	0 0 1 1 0;	83
35	0 1 0 0 0;	84
36	1 0 0 0 0;	85
37	1 1 1 1 1	86
38	];	87
39		88
40	D{4} = [	89
41	1 1 1 1 0;	90
42	0 0 0 0 1;	91
43	0 0 0 1 0;	92
44	0 0 1 1 0;	93
45	0 1 0 1 0;	94
46	1 1 1 1 1;	95
47	0 0 0 1 0	96
48	];	97
49		98
		99
100	D{10} = [	
101	0 1 1 1 0;	
102	1 0 0 0 1;	
103	1 0 0 0 1;	
104	0 1 1 1 1;	
105	0 0 0 0 1;	
106	0 0 0 0 1;	
107	0 1 1 1 0	
108	];	
109		
110	%% Convert patterns to bipolar form	
111	P = zeros(35,10);	
112		
113	for k = 1:10	
114	temp = D{k};	
115	temp(temp == 0) = -1;	
116	P(:,k) = temp(:);	
117	end	
118		
119	%% Calculate Hopfield weight matrix	
120	W = P * P';	
121		
122	% Remove self-connections	
123	W(1:36:end) = 0;	
124		
125	%% Test digit 8	
126	testDigit = 8;	
127	x = P(:,testDigit);	
128		
129	%% Add noise	
130	noiseLevel = 0.15;	
131		
132	rng(1);	
133		
134	noise = rand(size(x)) < noiseLevel;	
135		
136	xNoisy = x;	
137	xNoisy(noise) = -xNoisy(noise);	
138		
		D{5} = [
		1 1 1 1 1;
		1 0 0 0 0;
		1 0 0 0 0;
		1 1 1 1 0;
		0 0 0 0 1;
		0 0 0 0 1;
		1 1 1 1 0
		];
		D{6} = [
		0 1 1 1 0;
		1 0 0 0 0;
		1 0 0 0 0;
		1 1 1 1 0;
		1 0 0 0 1;
		1 0 0 0 1;
		0 1 1 1 0
		];
		D{7} = [
		1 1 1 1 1;
		0 0 0 0 1;
		0 0 0 1 0;
		0 0 1 0 0;
		0 1 0 0 0;
		0 1 0 0 0;
		0 1 0 0 0
		];
		D{8} = [
		0 1 1 1 0;
		1 0 0 0 1;
		1 0 0 0 1;
		0 1 1 1 0;
		1 0 0 0 1;
		1 0 0 0 1;
		0 1 1 1 0
		];
		D{9} = [
		0 1 1 1 0;
		1 0 0 0 1;
		1 0 0 0 1;
		0 1 1 1 1;
		0 0 0 0 1;
		0 0 0 0 1;
		0 1 1 1 0
		];

```

139 %% Recall using Hopfield Network
140 xRecall = xNoisy;
141
142 for iteration = 1:20
143     oldState = xRecall;
144     for i = 1:35
145         net = W(i,:) * xRecall;
146         if net >= 0
147             xRecall(i) = 1;
148         else
149             xRecall(i) = -1;
150         end
151     end
152     if isequal(xRecall,oldState)
153         break;
154     end
155 end
156
157 %% Identify recalled digit
158 similarity = P' * xRecall;
159 [~,index] = max(similarity);
160 recalledDigit = index - 1;
161
162 fprintf('Original Digit = %d\n',testDigit);
163 fprintf('Recalled Digit = %d\n',recalledDigit);
164 fprintf('Number of Iterations = %d\n',iteration);
165
166 %% Display results
167 figure;
168 subplot(1,3,1);
169 imagesc(reshape(x,7,5));
170 axis image;
171 axis off;
172 title('Original Digit');
173
174 subplot(1,3,2);
175 imagesc(reshape(xNoisy,7,5));
176 axis image;
177 axis off;
178 title('Noisy Digit');
179
180 subplot(1,3,3);
181 imagesc(reshape(xRecall,7,5));
182 axis image;
183 axis off;
184 title('Recalled Digit');
185
186 colormap(gray);

```



```

Original Digit = 8
Recalled Digit = 7
Number of Iterations = 2
>>

```